

Skid-Steer Rover: Dynamics Derivation and LQT Path-Following Controller

September 22, 2026

Abstract

This document derives a planar nonlinear dynamics model of a four-wheel skid-steer rover and describes the path-following controller built on it. The controller receives a reference path with heading and speed (the format expected from guidance and navigation), computes feedforward wheel torques by inverting the dynamics, and closes the loop with a linear-quadratic regulator on the lateral error states plus a proportional speed loop. All equations here match the MATLAB implementation, and the sign conventions are stated explicitly because they are easy to get wrong.

Contents

1	Overview	3
2	Rover dynamics	3
2.1	Frames, states, and sign conventions	3
2.2	Parameters	4
2.3	Translational dynamics	4
2.4	Yaw dynamics	5
2.5	Lateral slip (scrub) moment	5
2.6	Kinematics and full model	6
2.7	Modeling assumptions	6
3	Control design	6
3.1	Decoupling into speed and yaw channels	6
3.2	Reference trajectory	7
3.3	Feedforward torques	7
3.4	Error coordinates	7
3.5	LQR for the lateral loop	8
3.6	Speed loop	9

3.7	Complete control law	10
4	Implementation and results	10
4.1	Tracking a random path	10
4.2	Feedback test: recovery from initial offsets	11
4.3	Verification note	12
5	Limitations and next steps	12

1 Overview

The goal is to drive the rover to a target given in GPS coordinates by following a path supplied by the guidance and navigation team. The path is assumed to provide, along its length, the position, the *tangent heading* (course direction), and the commanded speed.

The rover has four independently driven wheels but cannot steer: it turns by driving the left and right sides at different torques. The design has four layers:

1. a nonlinear plant model (Section 2);
2. a decoupling of the four wheel torques into a *sum* torque (speed) and a *difference* torque (yaw) (Section 3.1);
3. feedforward torques computed by inverting the model along the reference (Section 3.3);
4. feedback: an LQR on the lateral error states and a proportional loop on speed (Sections 3.4–3.6).

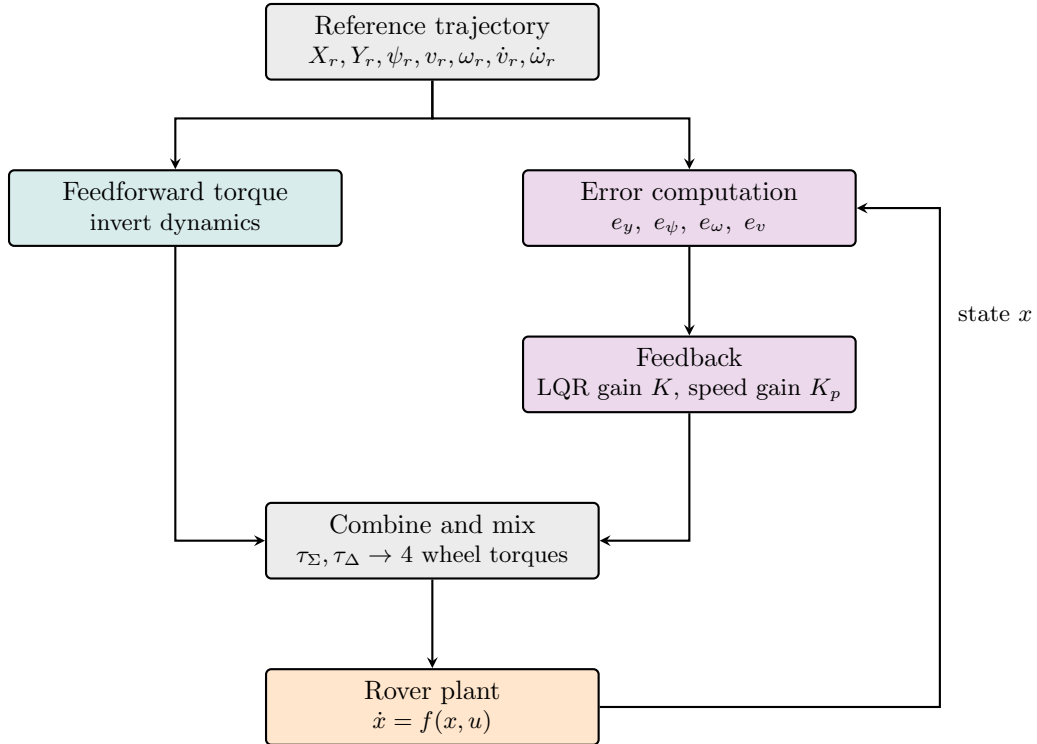


Figure 1: Control architecture. The left branch uses only the reference (open loop); the right branch uses the measured state.

2 Rover dynamics

2.1 Frames, states, and sign conventions

The inertial frame is North-East-Down (NED): X points North and Y points East (the down axis is unused for planar motion). The body frame has x_B forward, y_B to the left, and z_B up. The

state is

$$x = [X \ Y \ v \ \psi \ \omega]^\top, \quad (1)$$

where (X, Y) is the position of the center of gravity (m), v is the forward speed (m/s), ψ is the heading (rad), and ω is the yaw rate (rad/s).

Two different rotational conventions appear, and their sign difference matters:

- ω follows the right-hand rule about $+z_B$: **positive ω turns the nose to the left** (counter-clockwise seen from above).
- ψ is a *compass bearing*: $\psi = 0$ is North and it increases **clockwise** ($\psi = 90^\circ$ is East). A left turn therefore *decreases* ψ .

Consequently

$$\dot{\psi} = -\omega. \quad (2)$$

The four inputs are the wheel torques $\tau_{RL}, \tau_{FL}, \tau_{RR}, \tau_{FR}$ (rear/front, left/right). Only the side totals appear in the dynamics:

$$\tau_L = \tau_{RL} + \tau_{FL}, \quad \tau_R = \tau_{RR} + \tau_{FR}. \quad (3)$$

2.2 Parameters

Symbol	Meaning	Value	Units
m	rover mass	20	kg
r	wheel radius	0.10	m
d	track width	0.25	m
I_z	yaw moment of inertia	2.0	kg m ²
I_w	inertia of one wheel	0.01	kg m ²
C_{rr}	rolling resistance coefficient	0.02	–
μ	lateral (sliding) friction coefficient	0.6	–
a, b	CG-to-front/rear-axle distance	0.15, 0.15	m
ε	slip-velocity smoothing constant	0.03	m/s
g	gravity	9.81	m/s ²

Table 1: Model parameters. μ and C_{rr} are estimates and should be identified for the real wheels and terrain.

2.3 Translational dynamics

Let F_i be the longitudinal ground force on wheel i and $\dot{\omega}_{w,i}$ its spin acceleration. The wheel spin dynamics are $I_w \dot{\omega}_{w,i} = \tau_i - r F_i$, so $F_i = (\tau_i - I_w \dot{\omega}_{w,i})/r$. Assuming no longitudinal slip, a wheel's spin rate is tied to its ground speed, $r \omega_{w,i} = v_i$. The side speeds are

$$v_L = v - \frac{d}{2}\omega, \quad v_R = v + \frac{d}{2}\omega, \quad (4)$$

(positive ω means the right side is faster, which curves the rover left). Newton's second law with a rolling-resistance force F_{rr} opposing motion gives

$$m\dot{v} = \sum_{i=1}^4 F_i - F_{rr}. \quad (5)$$

Summing over the four wheels, the $\pm \frac{d}{2}\dot{\omega}$ terms cancel between sides and $\sum_i \dot{\omega}_{w,i} = 4\dot{v}/r$, so

$$\left(m + \frac{4I_w}{r^2}\right)\dot{v} = \frac{\tau_L + \tau_R}{r} - F_{rr}. \quad (6)$$

Define the effective mass $M_{\text{eff}} = m + 4I_w/r^2$ (the wheel inertia, reflected through the wheel radius, adds to the mass). Rolling resistance is modeled as Coulomb friction, $F_{rr} = C_{rr} mg \text{sign}(v)$:

$$\dot{v} = \frac{1}{M_{\text{eff}}} \left(\frac{\tau_L + \tau_R}{r} - C_{rr} mg \text{sign}(v) \right). \quad (7)$$

Note that this friction has a constant magnitude (no speed-dependent drag), so the model has no terminal velocity: a constant net torque produces a constant acceleration.

2.4 Yaw dynamics

The right wheels sit at $y_B = -d/2$ and the left wheels at $y_B = +d/2$. A forward force F at lateral position y_B produces a yaw moment $-y_B F$ about z_B , so the drive forces produce

$$M_{\text{drive}} = \frac{d}{2} (F_R - F_L), \quad (8)$$

where F_R, F_L are the total forces on each side. From $F_{\text{side}} = (\tau_{\text{side}} - 2I_w \dot{\omega}_{w,\text{side}})/r$ and $\dot{\omega}_{w,R/L} = (\dot{v} \pm \frac{d}{2}\dot{\omega})/r$,

$$F_R - F_L = \frac{\tau_R - \tau_L}{r} - \frac{2I_w d}{r^2} \dot{\omega}, \quad M_{\text{drive}} = \frac{d}{2r} (\tau_R - \tau_L) - \frac{I_w d^2}{r^2} \dot{\omega}. \quad (9)$$

The second term is the spin inertia of the wheels resisting the yaw acceleration; it is moved to the left-hand side of $I_z \dot{\omega} = M_{\text{drive}} + M_{\text{slip}}$, yielding an effective yaw inertia $J_{\text{yaw}} = I_z + I_w d^2/r^2$.

2.5 Lateral slip (scrub) moment

A skid-steer vehicle cannot yaw without its wheels sliding sideways. A point at longitudinal offset x_B from the CG has lateral velocity ωx_B due to rotation. The front wheels ($x_B = +a$) slide sideways at ωa and the rear wheels ($x_B = -b$) at $-\omega b$. Sliding friction opposes this motion. To avoid the discontinuity of a pure sign function, it is regularized with $\tanh$:

$$F_{y,f} = -\mu N_f \tanh\left(\frac{\omega a}{\varepsilon}\right), \quad F_{y,r} = -\mu N_r \tanh\left(\frac{-\omega b}{\varepsilon}\right), \quad (10)$$

with static normal loads per wheel (weight split by axle position)

$$N_f = \frac{b}{a+b} \frac{mg}{2}, \quad N_r = \frac{a}{a+b} \frac{mg}{2}. \quad (11)$$

The moment of these lateral forces about the CG, summed over the two wheels on each axle, is $x_B F_y$ per wheel:

$$M_{\text{slip}}(\omega) = 2aF_{y,f} - 2bF_{y,r} = -2\mu \left[aN_f \tanh\left(\frac{a\omega}{\varepsilon}\right) + bN_r \tanh\left(\frac{b\omega}{\varepsilon}\right) \right]. \quad (12)$$

This always opposes ω and is the physical damping that stops the rover from spinning forever after a turn command. Its slope at $\omega = 0$ is

$$M'_{\text{slip}}(0) = -\frac{2\mu}{\varepsilon}(a^2 N_f + b^2 N_r) \approx -88.3 \text{ N m s/rad}, \quad (13)$$

which is very steep because ε is small. This produces a fast yaw-rate time constant ($\approx 23 \text{ ms}$) and is the reason explicit ODE solvers need small steps or loose tolerances on this model.

The yaw dynamics are therefore

$$\dot{\omega} = \frac{1}{J_{\text{yaw}}} \left(\frac{d}{2r}(\tau_R - \tau_L) + M_{\text{slip}}(\omega) \right). \quad (14)$$

2.6 Kinematics and full model

$$\begin{cases} \dot{X} = v \cos \psi, & \dot{v} = \frac{1}{M_{\text{eff}}} \left(\frac{\tau_L + \tau_R}{r} - C_{rr} m g \text{sign}(v) \right), \\ \dot{Y} = v \sin \psi, & \dot{\omega} = \frac{1}{J_{\text{yaw}}} \left(\frac{d}{2r}(\tau_R - \tau_L) + M_{\text{slip}}(\omega) \right), \\ \dot{\psi} = -\omega. \end{cases} \quad (15)$$

2.7 Modeling assumptions

- Flat ground; no roll or pitch, so the static normal loads are constant (no load transfer).
- No longitudinal wheel slip; wheel spin is kinematically tied to ground speed.
- Rolling resistance is Coulomb-type and acts only longitudinally.
- Ideal torque actuators: no motor dynamics, torque limits, or saturation.
- The slip friction is regularized with $\tanh$; the smoothing constant ε sets how stiff the yaw damping is.

3 Control design

3.1 Decoupling into speed and yaw channels

Define the virtual controls

$$\tau_{\Sigma} = \tau_L + \tau_R, \quad \tau_{\Delta} = \tau_R - \tau_L. \quad (16)$$

Then $\dot{v}$ depends on the inputs *only* through τ_{Σ} , and $\dot{\omega}$ only through τ_{Δ} . The four-input, five-state problem splits into a speed loop and a yaw loop that interact only through the kinematics. The physical torques are recovered as

$$\tau_L = \frac{1}{2}(\tau_{\Sigma} - \tau_{\Delta}), \quad \tau_R = \frac{1}{2}(\tau_{\Sigma} + \tau_{\Delta}), \quad (17)$$

and each side torque is split equally between its front and rear wheel. Because there are only two effective inputs, the controller can regulate at most two independent quantities: speed, and one lateral quantity obtained through steering. The rover is nonholonomic, so X and Y cannot be commanded independently.

3.2 Reference trajectory

The reference is described along the path by arc length s . Let $\kappa(s)$ be the path curvature, defined as the rate of change of the tangent heading with distance, $\kappa = d\psi_r/ds$ ($\kappa > 0$ is a right turn in the compass convention). Then

$$\psi_r(s) = \psi_0 + \int_0^s \kappa d\sigma, \quad X_r(s) = \int_0^s \cos \psi_r d\sigma, \quad Y_r(s) = \int_0^s \sin \psi_r d\sigma. \quad (18)$$

A speed profile $v_r(t)$ maps time to arc length, $s(t) = \int_0^t v_r d\tau$. We use a trapezoidal profile: accelerate at $a_{\max}$ to the cruise speed v_c , cruise, then decelerate to rest at the end of the path, with

$$t_a = \frac{v_c}{a_{\max}}, \quad d_a = \frac{v_c^2}{2a_{\max}}, \quad t_c = \frac{S - 2d_a}{v_c}, \quad (19)$$

where S is the total path length (if $S < 2d_a$ the profile is triangular).

Along the path the reference yaw rate follows from $\dot{\psi}_r = v_r \kappa$ and $\dot{\psi} = -\omega$:

$$\omega_r = -v_r \kappa(s), \quad \dot{\omega}_r = -(\dot{v}_r \kappa + v_r^2 \kappa'(s)), \quad (20)$$

where the second expression is the time derivative of the first using $\dot{s} = v_r$. The reference function is evaluated fresh at each time t , independently of where the rover actually is.

For testing, a random smooth path is generated by summing a few sinusoids in arc length and tapering the curvature to zero at both ends,

$$\kappa(s) = \sin\left(\frac{\pi s}{S}\right) \sum_{k=1}^K \frac{\kappa_{\max}}{K} \sin\left(\frac{2\pi k s}{S} + \phi_k\right), \quad (21)$$

with random phases ϕ_k and $\kappa_{\max} = 1/R_{\min}$, then integrating numerically. This is only a stand-in for the path supplied by guidance and navigation.

3.3 Feedforward torques

If the rover were exactly on the reference, the required torques follow by solving the model for the reference accelerations:

$$\boxed{\tau_{\Sigma}^{\text{ff}} = r \left(M_{\text{eff}} \dot{v}_r + C_{rr} m g \text{sign}(v_r) \right), \quad \tau_{\Delta}^{\text{ff}} = \frac{2r}{d} \left(J_{\text{yaw}} \dot{\omega}_r - M_{\text{slip}}(\omega_r) \right).} \quad (22)$$

This handles the known reference motion (acceleration, rolling resistance, and the slip damping at the reference yaw rate) so that feedback only has to correct deviations.

3.4 Error coordinates

Let $p = (X, Y)$ and $p_r = (X_r, Y_r)$, and define the reference tangent and left-pointing normal

$$\mathbf{t}_r = \begin{bmatrix} \cos \psi_r \\ \sin \psi_r \end{bmatrix}, \quad \mathbf{n}_r = \begin{bmatrix} \sin \psi_r \\ -\cos \psi_r \end{bmatrix}, \quad (23)$$

with components ordered $(X, Y) = (\text{North}, \text{East})$. For $\psi_r = 0$ (heading North), $\mathbf{t}_r$ points North and $\mathbf{n}_r$ points West, i.e. to the left of the direction of travel. The tracking errors are

$$\begin{aligned} e_y &= (p - p_r) \cdot \mathbf{n}_r = (X - X_r) \sin \psi_r - (Y - Y_r) \cos \psi_r \quad (\text{cross-track; } > 0 \text{ means left of the path}), \\ e_\psi &= \text{wrap}(\psi - \psi_r) \quad (\text{heading error, wrapped to } [-\pi, \pi]), \\ e_\omega &= \omega - \omega_r, \quad e_v = v - v_r. \end{aligned} \tag{24}$$

Error kinematics. Differentiating e_y , with $\dot{p} = v (\cos \psi, \sin \psi)^\top$, $\dot{p}_r = v_r \mathbf{t}_r$, and $\dot{\mathbf{n}}_r = \dot{\psi}_r \mathbf{t}_r$:

$$\dot{e}_y = v (\cos \psi, \sin \psi) \cdot \mathbf{n}_r + e_s \dot{\psi}_r = -v \sin e_\psi + e_s \dot{\psi}_r, \tag{25}$$

where $e_s = (p - p_r) \cdot \mathbf{t}_r$ is the along-track error. Linearizing for small errors and neglecting the product $e_s \dot{\psi}_r$ (small when the along-track error is small),

$$\dot{e}_y \approx -v_r e_\psi. \tag{26}$$

The minus sign follows from the choice of $\mathbf{n}_r$ pointing left: heading clockwise of the path ($e_\psi > 0$) moves the rover to the right, decreasing e_y . The heading error is exact:

$$\dot{e}_\psi = \dot{\psi} - \dot{\psi}_r = -\omega + \omega_r = -e_\omega. \tag{27}$$

Yaw-rate error dynamics. Subtracting the reference yaw equation from the actual one, and writing $\Delta u = \tau_\Delta - \tau_\Delta^{\text{ff}}$ for the feedback correction,

$$\dot{e}_\omega = \frac{1}{J_{\text{yaw}}} \left(\frac{d}{2r} \Delta u + M_{\text{slip}}(\omega) - M_{\text{slip}}(\omega_r) \right) \approx \frac{M'_{\text{slip}}(\omega_r)}{J_{\text{yaw}}} e_\omega + \frac{d/2r}{J_{\text{yaw}}} \Delta u, \tag{28}$$

using a first-order Taylor expansion of M_{slip} about ω_r . Because M'_{slip} is largest at $\omega = 0$ and shrinks rapidly for larger $|\omega|$ (the tanh saturates), we linearize once about the straight-line reference $\omega_r = 0$.

Linear error model. With $e = [e_y, e_\psi, e_\omega]^\top$, $\dot{e} = A e + B \Delta u$ where

$$A = \begin{bmatrix} 0 & -v_c & 0 \\ 0 & 0 & -1 \\ 0 & 0 & M'_{\text{slip}}(0)/J_{\text{yaw}} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ \frac{d}{2r J_{\text{yaw}}} \end{bmatrix}. \tag{29}$$

For the parameters in Table 1 and $v_c = 0.5 \text{ m/s}$: $J_{\text{yaw}} = 2.0625 \text{ kg m}^2$, $d/(2r) = 1.25$, $M'_{\text{slip}}(0)/J_{\text{yaw}} = -42.81 \text{ s}^{-1}$, and $B_3 = 0.606$. The pair (A, B) is controllable for $v_c \neq 0$.

3.5 LQR for the lateral loop

We choose $\Delta u = -K e$ to minimize

$$J = \int_0^\infty (e^\top Q e + R \Delta u^2) dt. \tag{30}$$

A and B are Jacobians of the dynamics and come from the physics. $Q \succeq 0$ and $R > 0$ are *design weights* that trade tracking error against torque effort; they are tuned, not derived. The optimal gain is $K = R^{-1}B^\top P$, where $P \succ 0$ solves the algebraic Riccati equation

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0. \quad (31)$$

The equation is solved with the Hamiltonian method: form $H = \begin{bmatrix} A & -BR^{-1}B^\top \\ -Q & -A^\top \end{bmatrix}$, take the eigenvectors belonging to its n stable eigenvalues and stack them as $\begin{bmatrix} X_1 \\ X_2 \end{bmatrix}$, then $P = X_2 X_1^{-1}$. This requires only base MATLAB (`eig`), not the Control System Toolbox.

With $Q = \text{diag}(5, 20, 1)$ and $R = 0.05$:

$$P \approx \begin{bmatrix} 33.40 & -35.77 & 0.825 \\ -35.77 & 119.07 & -2.755 \\ 0.825 & -2.755 & 0.0756 \end{bmatrix}, \quad K \approx \begin{bmatrix} 10.00 & -33.40 & 0.916 \end{bmatrix}, \quad (32)$$

so the feedback is $\Delta u = -10e_y + 33.4e_\psi - 0.916e_\omega$ (a left-of-path error commands a right turn; a clockwise heading error commands a left turn). The closed-loop poles $A - BK$ are -42.9 and $-0.235 \pm 0.124j$: one fast mode (the yaw-slip damping) and a slow, well-damped pair ($\zeta \approx 0.88$) governing path convergence with a time constant of a few seconds. Increasing the first entry of Q speeds up cross-track recovery at the cost of torque effort.

Relation to LQT. A finite-horizon linear-quadratic tracker for a reference produces a time-varying feedback gain plus a feedforward term from a backward co-state equation. Here the tracking problem is instead reduced to regulation: the nonlinear inverse dynamics supply the feedforward (Section 3.3) and a constant-gain LQR regulates the error about the reference. This is the common practical form and is exact for the reference motion; the linearization (and hence K) is fixed at cruise speed and zero yaw rate.

3.6 Speed loop

Since τ_Σ^{ff} cancels rolling resistance and produces $\dot{v}_r$ exactly at $v = v_r$, subtracting the reference speed equation from the actual one and writing $\tau_\Sigma = \tau_\Sigma^{\text{ff}} + \tau_\Sigma^{\text{fb}}$ leaves

$$\dot{e}_v = \frac{\tau_\Sigma^{\text{fb}}}{r M_{\text{eff}}}. \quad (33)$$

With proportional feedback $\tau_\Sigma^{\text{fb}} = -K_p e_v$ this is the first-order decay $\dot{e}_v = -\frac{K_p}{r M_{\text{eff}}} e_v$, i.e. $e_v(t) = e_v(0) e^{-t/\tau_v}$ with time constant $\tau_v = r M_{\text{eff}}/K_p$. Choosing the desired time constant gives the gain directly,

$$K_p = \frac{r M_{\text{eff}}}{\tau_v}. \quad (34)$$

For $r M_{\text{eff}} = 0.1 \times 24 = 2.4$ and $\tau_v = 0.5$ s, $K_p = 4.8$.

3.7 Complete control law

At each time t , the controller evaluates the reference $(X_r, Y_r, \psi_r, v_r, \omega_r, \dot{v}_r, \dot{\omega}_r)(t)$, forms $(e_y, e_\psi, e_\omega, e_v)$ from the measured state, and commands

$$\boxed{\tau_\Sigma = \tau_\Sigma^{\text{ff}} - K_p e_v, \quad \tau_\Delta = \tau_\Delta^{\text{ff}} - K [e_y, e_\psi, e_\omega]^\top,} \quad (35)$$

which are then mixed into the four wheel torques as in Section 3.1.

4 Implementation and results

File	Role
<code>rover_dynamics.m</code>	nonlinear plant $\dot{x} = f(x, u)$
<code>random_path_geom.m</code>	random smooth test path (curvature integration)
<code>reference_trajectory.m</code>	time-indexed reference and speed profile
<code>yaw_slip_moment.m</code>	$M_{\text{slip}}(\omega)$
<code>feedforward_torque.m</code>	$\tau_\Sigma^{\text{ff}}, \tau_\Delta^{\text{ff}}$
<code>care_solve.m</code>	Riccati solver (Hamiltonian method)
<code>lateral_lqr_gain.m</code>	builds A, B ; returns K
<code>speed_loop_gain.m</code>	$K_p = rM_{\text{eff}}/\tau_v$
<code>rover_controller.m</code>	full control law
<code>main.m, monte_carlo.m</code>	closed-loop simulation and Monte Carlo

Table 2: MATLAB implementation map.

4.1 Tracking a random path

Figure 2 shows the closed loop following a random 18 m path at $v_c = 0.5$ m/s. Over $N = 20$ random paths (nominal model, starting on the reference), the maximum cross-track error had a mean of 0.9 mm (worst 1.3 mm), the maximum heading error a mean of 0.014° (worst 0.021°), and the final position error a mean of 1.7 mm (worst 3.9 mm).

These numbers are deliberately optimistic. The simulated plant is identical to the model used to compute the feedforward, so the feedforward alone nearly tracks the path and feedback has little to do. They demonstrate that the reference generation, inverse dynamics, and sign conventions are consistent; they do not predict real-hardware accuracy.

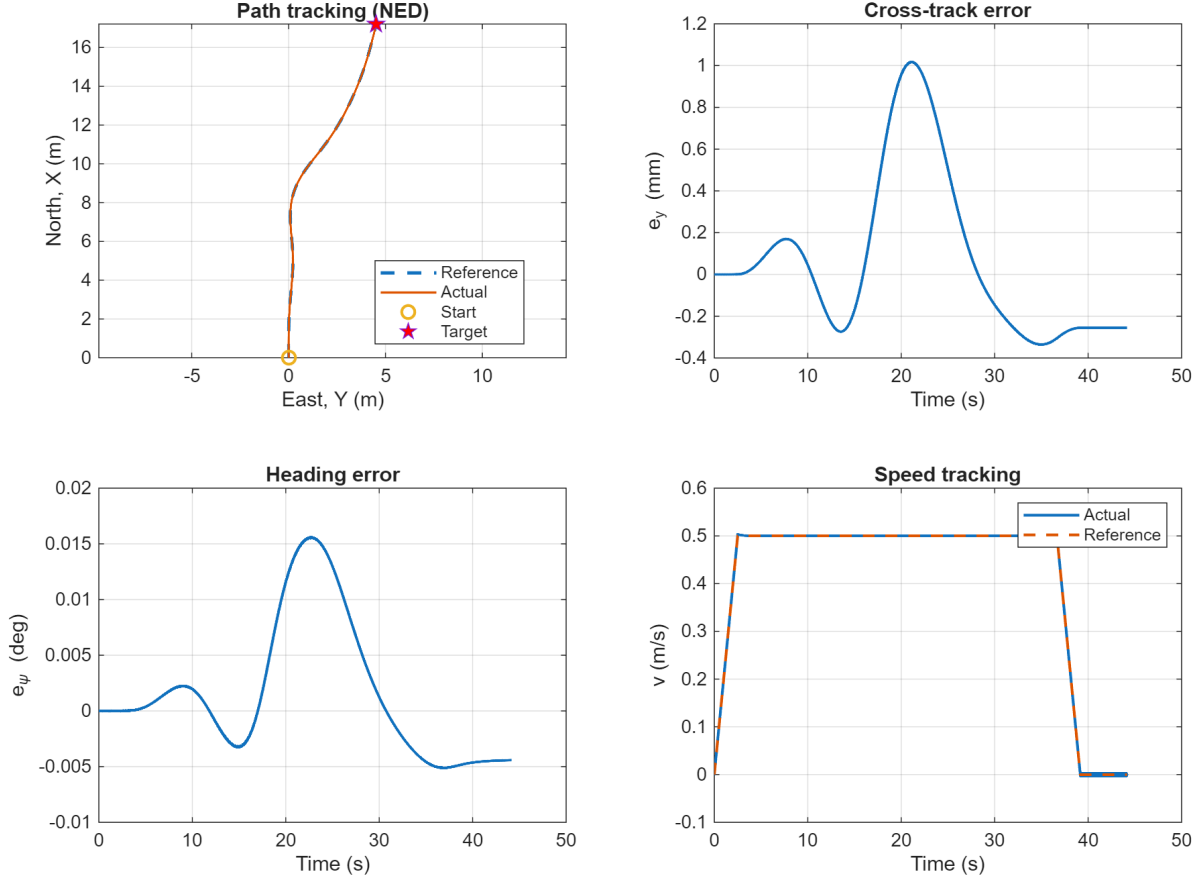


Figure 2: Closed-loop tracking of a randomly generated path. Cross-track error stays around a millimeter and speed follows the trapezoidal profile.

4.2 Feedback test: recovery from initial offsets

A stronger test of the feedback loop is to start the rover off the path. Figure 3 shows recovery from initial lateral offsets of 0.3–0.5 m and initial heading offsets of -20° and $+15^\circ$. The cross-track error decays to millimeters within 20–30 s, and heading error within roughly 15 s, with mild overshoot consistent with the lightly damped slow pole pair ($\zeta \approx 0.88$). Recovery time is set by the choice of Q and R and is intentionally gentle here.

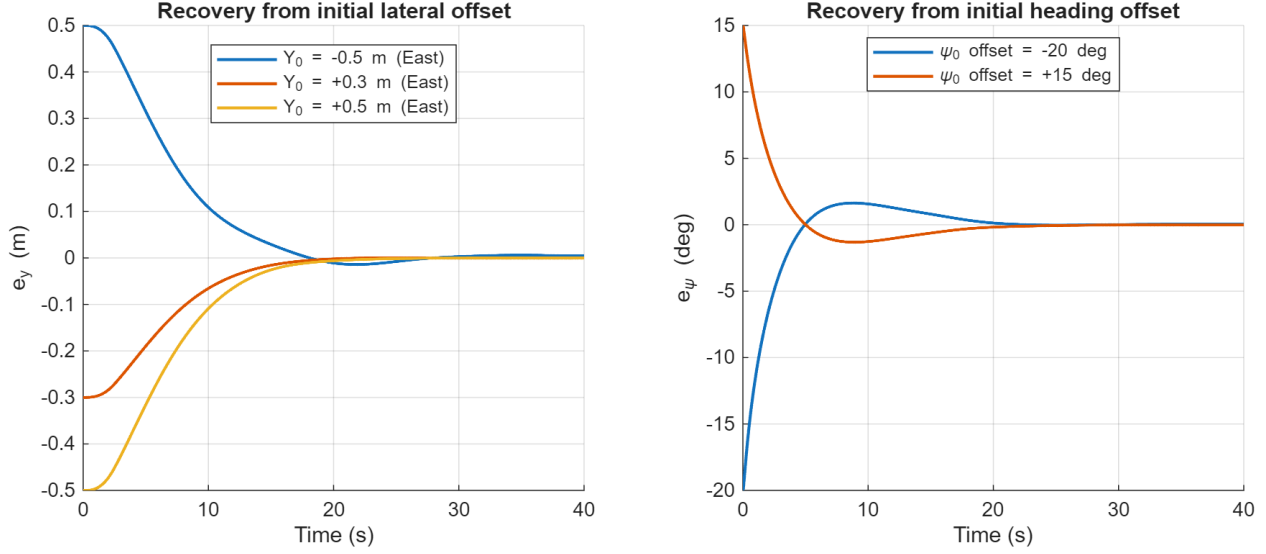


Figure 3: Recovery from initial lateral offset (left) and heading offset (right). In the left plot, a start west of a north-heading path corresponds to $e_y > 0$ (left of the path).

4.3 Verification note

The sign of the $\dot{e}_y$ term in the error model is easy to get wrong, and a wrong sign is masked whenever the rover starts on the path, because good feedforward keeps the errors near zero. The recovery-from-offset test above is what exposes it: with $\dot{e}_y = +v e_\psi$ instead of $-v e_\psi$ the closed loop has an unstable pole and a 0.3 m offset grows rather than decays. Any future change to the error definitions should be re-checked with an offset test, not only with on-path tracking.

5 Limitations and next steps

- **Fixed-gain linearization.** K is computed at $v_c = 0.5$ m/s and $\omega_r = 0$. Faster speeds or tighter turns may call for gain scheduling on (v_r, κ) .
- **No along-track feedback.** The speed loop regulates v , not progress s ; a disturbance that puts the rover behind schedule is not made up. The term $e_s \dot{\psi}_r$ dropped from $\dot{e}_y$ is likewise assumed small.
- **Heading convention.** The controller assumes the heading supplied by guidance and navigation is the path's tangent (course) direction. If it is instead a line-of-sight bearing to the target, the tangent must be derived from consecutive waypoints.
- **Ideal actuation and sensing.** No torque saturation, motor dynamics, or state estimation; the controller uses the true state.
- **Parameter uncertainty.** μ , C_{rr} , ε , and I_z are estimates. A Monte Carlo over parameter uncertainty (not only path geometry) is the natural next robustness test.
- **Numerical stiffness.** The fast yaw-slip pole (≈ -43 s $^{-1}$) forces small explicit-solver steps; loose tolerances (e.g. `RelTol`= 10^{-2}) give a large speedup with negligible accuracy loss here.